Laboratorio 1 BDA

Plataforma de Logística y Seguimiento de Envíos

Integrantes
Matías Leiva
Mario Nova
Benjamín Bastías
Patricio Durán
Héctor Berguño
Felipe Bobadilla



github.com/bbastias7/Laboratorio-1-BDA

Cada paquete debe poder rastrearse

Introducción: el problema, el objetivo y quiénes lo usan

El problema

Coordinar centros de distribución, vehículos, repartidores y rutas, y que el cliente sepa en todo momento dónde está su paquete.

Nuestro objetivo

Una plataforma web con API REST y PostgreSQL, donde las reglas del negocio viven dentro de la base de datos.

Tres roles

Cliente
crea y rastrea envíos

Despachador
asigna rutas y revisa reportes

Repartidor
confirma entregas y fallas

Avance: requisitos implementados

Cada fila indica dónde vive en el repositorio

Requisito del enunciado	Estado	Dónde
CRUD de todas las entidades	✓	9 controladores · /api/*
JWT y roles por usuario	✓	SecurityConfig · @PreAuthorize
SP 1: asignar ruta	✓	asignar_ruta()
SP 2: entrega fallida	✓	registrar_entrega_fallida()
Trigger: capacidad del vehículo	✓	check_route_capacity()
Trigger: auditoría de destino	✓	audit_destination_change()
Vista materializada	✓	mv_rendimiento_entregas
Índices: código y estado	✓	idx_paquete_codigo · _estado
Rastreo por código	✓	GET /api/tracking/{codigo}
Reporte bajo el 85 %	✓	GET /api/reportes/alertas

Transversal	Dónde
Refresco diario de la vista	@Scheduled 03:00
5 índices, HASH en código	01-dbCreate.sql
9 triggers	04-triggers.sql
JWT + roles (RBAC)	SecurityConfig
README con instalación	README.md
Script y datos de prueba	01-dbCreate.sql · 05-loadData.sql

Etapa final: paginación y filtros en el CRUD · registro de clientes y personal · correcciones de la revisión

Tres roles coordinan el mismo paquete

Cada persona ve las operaciones que le corresponden

Cliente

Crear envío

Consultar sus paquetes

Rastrear y cambiar destino

Despachador

Asignar rutas

Administrar entidades

Revisar reportes y auditoría

Repartidor

Ver sus rutas

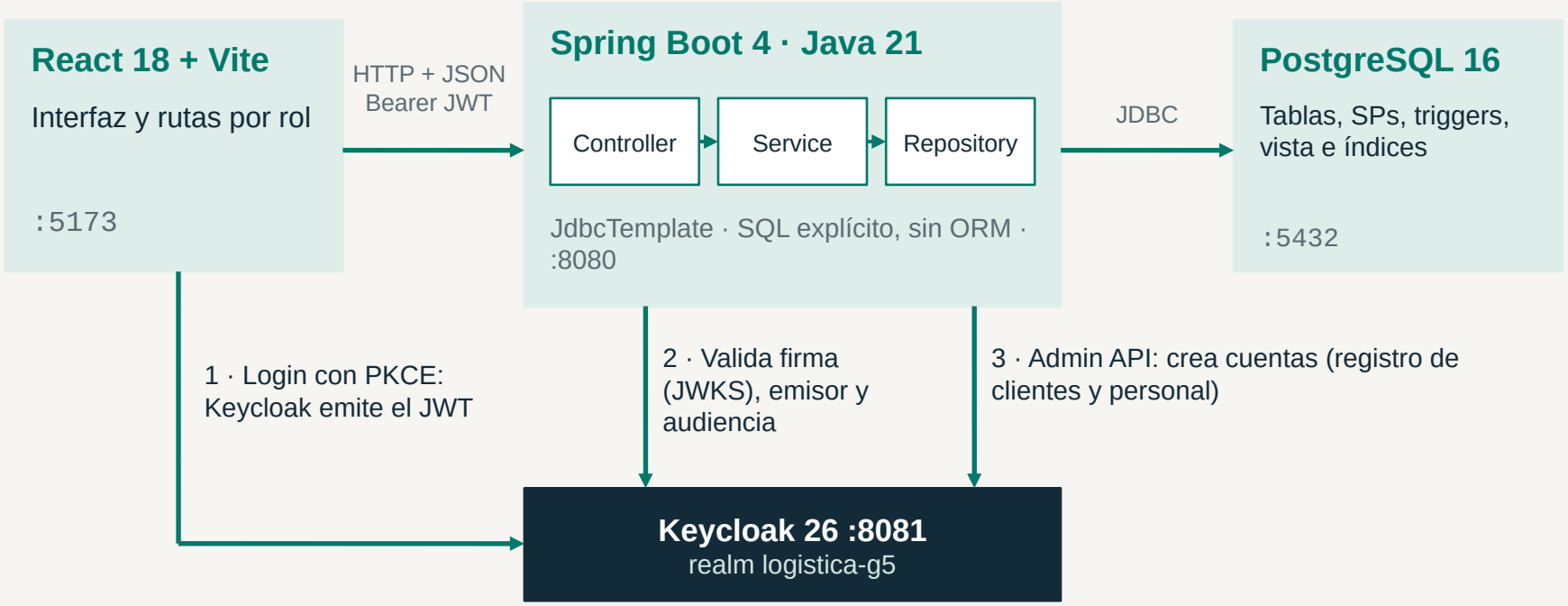
Confirmar entregas

Registrar fallas y motivos

Permisos por rol + filtros por propietario en la API

Cuatro componentes conectados por JWT

React presenta, Spring coordina, PostgreSQL conserva y valida, Keycloak identifica



Un clic recorre todas las capas

Ejemplo: registrar un envío y devolver su código

01

Formulario

React valida y convierte los decimales

02

Solicitud

POST /api/paquete JSON + Bearer JWT

03

Servicio

Dueño desde el JWT, estado Registrado, @Transactional

04

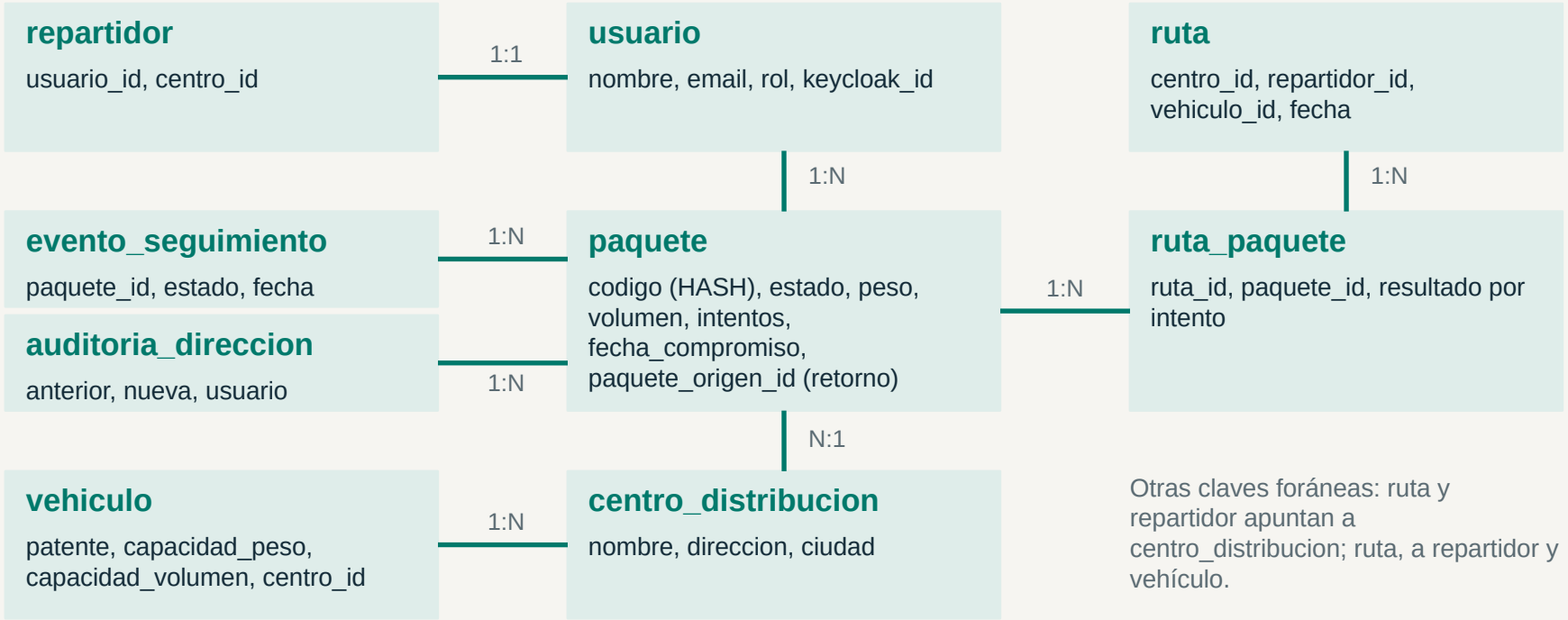
PostgreSQL

INSERT de paquete + evento inicial; el código lo genera la base

Respuesta: paquete creado (201) y código de seguimiento para React

El historial une paquetes, rutas y personas

Modelo relacional: 9 tablas y sus relaciones principales



Otras claves foráneas: ruta y repartidor apuntan a centro_distribucion; ruta, a repartidor y vehículo.

Índices y consultas clave

Qué se indexa, por qué, y la consulta de rastreo

Índice	Tipo	Para qué
`idx_paquete_codigo`	HASH	Rastreo: solo igualdad (codigo = ?)
`idx_paquete_estado`	B-tree	Filtros y listados por estado
`idx_evento_seguimiento_paquete_fecha`	B-tree (3 col.)	Línea de tiempo ordenada
`uq_ruta_paquete_pendiente`	Único parcial	Una asignación pendiente por paquete

Rastreo: GET /api/tracking/{codigo}

```
SELECT es.id, es.paquete_id, es.estado, es.descripcion, es.fecha
FROM evento_seguimiento es JOIN paquete p ON es.paquete_id = p.id
WHERE p.codigo = ? ORDER BY es.fecha ASC, es.id ASC
```

Antes, PaqueteService.rastrear() comprueba que el usuario pueda ver el paquete (esVisible).

Vista materializada: reportes baratos

mv_rendimiento_entregas, refrescada cada día sin bloquear lecturas

```
CREATE MATERIALIZED VIEW
  mv_rendimiento_entregas AS
SELECT r.centro_id, r.repartidor_id, ...
 count(*) FILTER(WHERE rp.resultado='Entregado'
   AND rp.fecha_resultado<=p.fecha_compromiso)
   ::integer AS entregas_a_tiempo,
 ... AS pct_a_tiempo, ... AS intentos_promedio
FROM ruta_paquete rp JOIN ruta r ON ...
GROUP BY r.centro_id, ..., r.repartidor_id, ...;
```

```
REFRESH MATERIALIZED VIEW CONCURRENTLY
  mv_rendimiento_entregas;
```

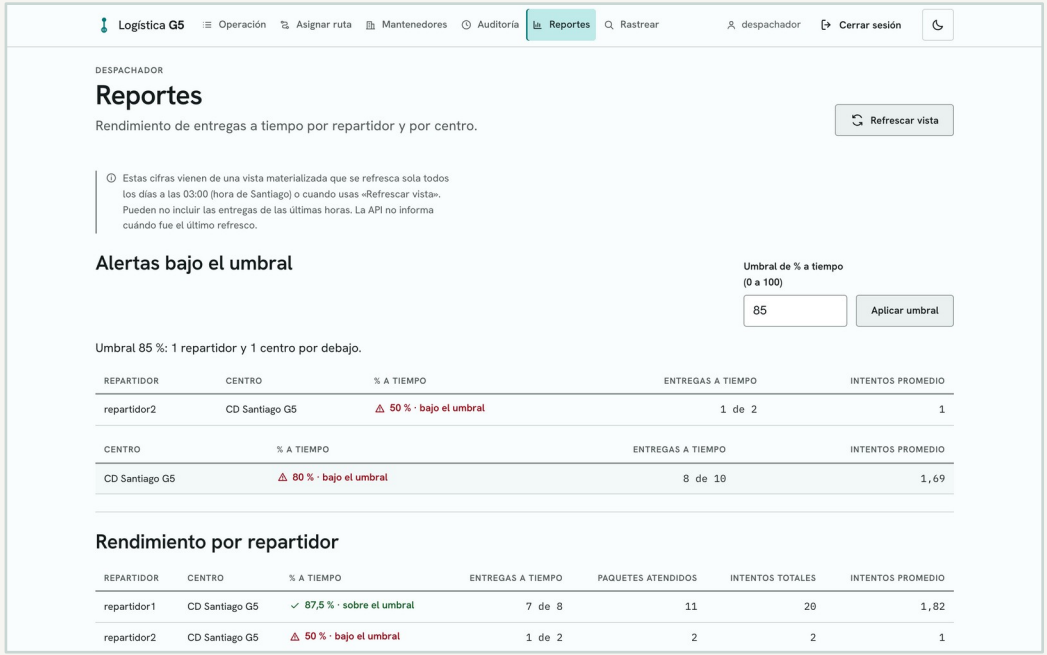
Refresco diario

@Scheduled a las 03:00 (America/Santiago) y botón "Refrescar vista".

```
@Scheduled(
  cron = "${REFRESH_CRON:
    0 0 3 * * *}",
  zone = "America/Santiago")
```

El reporte alerta bajo el 85 %

GET /api/reportes/alertas?umbral=85 cruza la vista materializada



Captura real de la base de demostración

80 %

CD Santiago G5: 8 entregas a tiempo de 10, bajo el umbral de 85 %

```
WHERE 100.0 *
  entregas_a_tiempo /
  NULLIF(entregas_totales,0)
  < ?
```

ReporteRepository · repartidoresBajo()

SP 1: asignar ruta en una transacción

asignar_ruta(centro, repartidor, vehículo, fecha, paquetes)

```
INSERT INTO ruta (centro_id, repartidor_id,
    vehiculo_id, fecha)
VALUES (p_centro_id, p_repartidor_id,
    p_vehiculo_id, COALESCE(p_fecha, now()))
RETURNING id INTO p_ruta_id;
-- The capacity trigger fires per row
INSERT INTO ruta_paquete (ruta_id, paquete_id)
SELECT p_ruta_id, x FROM unnest(p_paquetes) ...;
UPDATE paquete SET estado = 'En reparto', ...
 WHERE id = ANY (p_paquetes);
INSERT INTO evento_seguimiento
    (paquete_id, estado, descripcion, fecha)
SELECT x, 'En reparto',
    'Asignado a la ruta ' || p_ruta_id, ...
```

1. Crea la ruta
2. Asocia paquetes y vehículo; el trigger valida la capacidad
3. Cambia los paquetes a "En reparto"
4. Registra un evento por paquete

Si algo falla, todo se revierte.

Trigger 1: capacidad del vehículo

BEFORE INSERT en ruta_paquete · una comprobación por fila

```
CREATE TRIGGER route_capacity_before_insert
BEFORE INSERT ON ruta_paquete
FOR EACH ROW EXECUTE FUNCTION check_route_capacity();
-- dentro de check_route_capacity()
IF total_weight > max_weight OR total_volume > max_volume THEN
    RAISE EXCEPTION 'Route % exceeds vehicle capacity (weight % / %, volume % / %)',
        NEW.ruta_id, total_weight, max_weight, total_volume, max_volume
        USING ERRCODE = '23514';
END IF;
RETURN NEW;
```

NEW

Fila que se insertará

OR

Peso o volumen excedido

EXCEPTION

Rechaza y revierte todo

SP 2: entrega fallida y retorno

registrar_entrega_fallida: reprogramar o devolver es una decisión del procedimiento

1	**Reprogramado**	nueva fecha + 1 día
2	**Reprogramado**	nueva fecha + 1 día
3	**Reprogramado**	nueva fecha + 1 día
4	**Devuelto a origen**	crea paquete de retorno

```
v_pkg.intentos := v_pkg.intentos + 1;
IF v_pkg.intentos > 3 THEN
    UPDATE paquete SET intentos = v_pkg.intentos,
        estado = 'Devuelto a origen', ... WHERE id = p_paquete_id;
    INSERT INTO paquete (estado, peso, ..., paquete_origen_id)
    VALUES ('Registrado', v_pkg.peso, ..., p_paquete_id)
    RETURNING id INTO v_nuevo_id;
ELSE
    UPDATE paquete SET intentos = v_pkg.intentos, estado = 'Reprogramado',
        fecha_reprogramada = now() + interval '1 day' WHERE id = p_paquete_id;
END IF;
```

Trigger 2: cambiar el destino deja huella

La auditoría registra quién cambió qué y cuándo, y no se puede editar

```
-- audit_destination_change(): BEFORE UPDATE OF direccion_destino
IF OLD.despachado_at IS NOT NULL
   AND NEW.direccion_destino IS DISTINCT FROM OLD.direccion_destino THEN
    actor := NULLIF(current_setting('app.user_id',true), '')::integer;
    INSERT INTO auditoria_direccion
        (paquete_id, direccion_anterior, direccion_nueva, usuario_id, fecha)
    VALUES (OLD.id, OLD.direccion_destino, NEW.direccion_destino,
            actor, now());
END IF;
```

```
CREATE TRIGGER auditoria_inmutable BEFORE UPDATE OR DELETE
ON auditoria_direccion FOR EACH ROW EXECUTE FUNCTION proteger_auditoria();
```

Estados finales (Entregado, Devuelto a origen): cambio de destino bloqueado.

La API verifica identidad y acceso

JWT, permisos por rol y pertenencia del recurso

```
// SecurityConfig: emisor + audiencia
decoder.setJwtValidator(
  new DelegatingOAuth2TokenValidator<>(
    JwtValidators.createDefaultWithIssuer(issuer),
    validarAudiencia));
// PaqueteService.rastrear()
if (!paqueteRepository.esVisible(
        p.getPaqueteId(), u.rol(), u.uid())) {
    throw new AccessDeniedException(
        "Paquete de otro usuario");
}
```

1 JWT válido

firma, emisor y audiencia

2 Rol autorizado

@PreAuthorize por endpoint

3 Recurso permitido

esVisible: el paquete es del usuario

Conocer un código no concede acceso a un envío ajeno.

Registro de usuario con Keycloak

Registro público: solo clientes, con compensación si falla



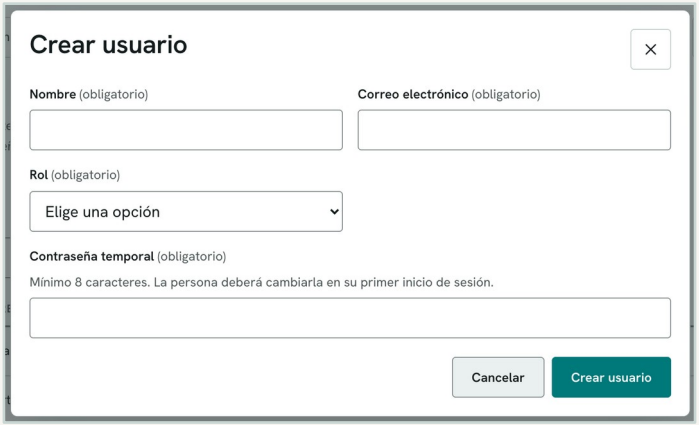
Captura real de /registro

1. React: formulario "Crear cuenta"
2. POST /api/auth/registro (público)
3. Keycloak Admin API: crea el usuario, rol cliente forzado
4. INSERT en usuario; si falla, se elimina en Keycloak

```
keycloakId = keycloak.crearUsuario(
    nombre, email, req.password(), ROL);
try {
    return usuarios.save(new UsuarioEntity(
        null, nombre, email, ROL, keycloakId));
} catch (RuntimeException e) {
    keycloak.eliminarUsuario(keycloakId);
```

Cuentas del personal: solo el despachador

Crear usuario genera la cuenta de Keycloak con una contraseña temporal



Captura real: Usuarios, Crear usuario (sin enviar)

/registro solo crea clientes y no elige rol: no hay escalada de privilegios.

1. Despachador: Usuarios, Crear usuario
2. Keycloak crea la cuenta; la clave temporal se cambia al ingresar
3. INSERT en usuario; si falla, se elimina la cuenta

```
keycloakId = keycloak.crearUsuario(
    nombre, email, req.password(),
    req.rol(), true);
try {
    return usuarioRepository.save(
        new UsuarioEntity(null, nombre, email,
            req.rol(), keycloakId));
} catch (RuntimeException e) {
    keycloak.eliminarUsuario(keycloakId);
```

CRUD con paginación y filtros

Los 9 recursos aceptan page y size, más un filtro simple



Captura real: Eventos de seguimiento (72 registros, página 1 de 5)

Sin page, el endpoint devuelve la lista completa. Con page, devuelve esa página y el total en X-Total-Count, expuesto por CORS.

```
GET /api/paquete
  ?page=0&size=10
  &estado=Registrado

200 OK
X-Total-Count: <total>
```

```
// Pagina.responder()
if (total != null) {
  ok.header(TOTAL_HEADER,
    String.valueOf(total));
}
```

Validar antes de enviar

Formato numérico, valor positivo y bloqueo del envío

```
export function parseDecimal(raw) {
  const s = String(raw ?? '').trim().replace(',', '.');
  return /^\d+(\.\d+)?$/.test(s) ? Number(s) : Number.NaN;
}

// ClienteNuevo.jsx · submit
const found = validate(values);
setErrors(found);
setServerError(null);
if (Object.keys(found).length) {
  // ... enfoque del campo con error omitido
  return;
}
```

0, -1, abc: no se guarda. 2,5: formato válido.

Demostración: del registro a la entrega



1 · Cliente crea el envío y recibe el código



2 · Despachador asigna la ruta: "En reparto"



3 · Repartidor marca la entrega realizada

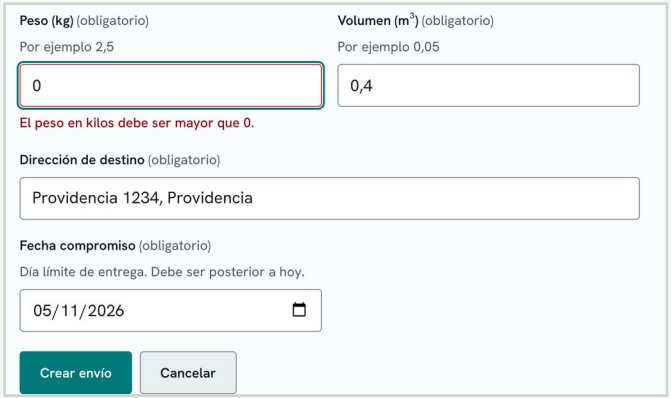


4 · Cliente rastrea: Entregado

Demostración: datos inválidos

Pruebas que se pueden solicitar durante la exposición

Pantalla / campo	Entrada de prueba	Resultado esperado
Crear envío / peso y volumen	0, -1, abc (por separado)	Error y sin crear envío
Crear envío / fecha	Hoy o una fecha anterior	Error: debe ser futura
Asignar ruta / carga total	Peso o volumen sobre el límite	Asignación rechazada
Crear envío / caso válido	2,5 kg + 0,4 m³ + fecha futura	Código de seguimiento



Captura real: peso 0 muestra el error y no envía

Corregir el dato y repetir el flujo válido

Contribuciones del equipo

Contribuciones y responsabilidades del equipo

Integrante	Aporte
Matías Leiva	Correcciones de la revisión (función de refresco, índice hash, auditoría inmutable, CORS), manual de instalación, frontend web, paginación y registro de usuarios
Mario Nova	Dump de demostración y restauración verificada; documentación del respaldo; revisión de rúbrica y enunciado con pruebas SQL, API y concurrencia
Benjamín Bastías	Estructura inicial del repositorio y CRUD en los controladores restantes
Patricio Durán	Ajustes de entidades y repositorios, CRUD básico, restricciones en la base de datos; backend y scripts de BD completos
Héctor Berguño	Responsabilidades de cierre: revisión de la entrega, preparación de la demostración web y exposición de seguridad, registro y paginación
Felipe Bobadilla	Script inicial de la base de datos, entidades, repositorios, servicios y controladores

Conclusiones y próximos pasos

Lo que el proyecto nos dejó

Reglas en la base de datos

Triggers y procedimientos no se pueden saltar desde la API.

Identidad no es autorización

JWT más validación de propiedad en cada recurso.

Frescura contra costo

La vista materializada abarata los reportes, con refresco diario.

Lecciones

Sin ORM el SQL es explícito; un README instalable es un requisito.

Próximos pasos: pg_cron para el refresco · rastreo público · notificaciones

Gracias. Preguntas

Repositorio

github.com/bbastias7/Laboratorio-1-BDA

README con instalación · database/ · backend/ · frontend/ · presentacion/